

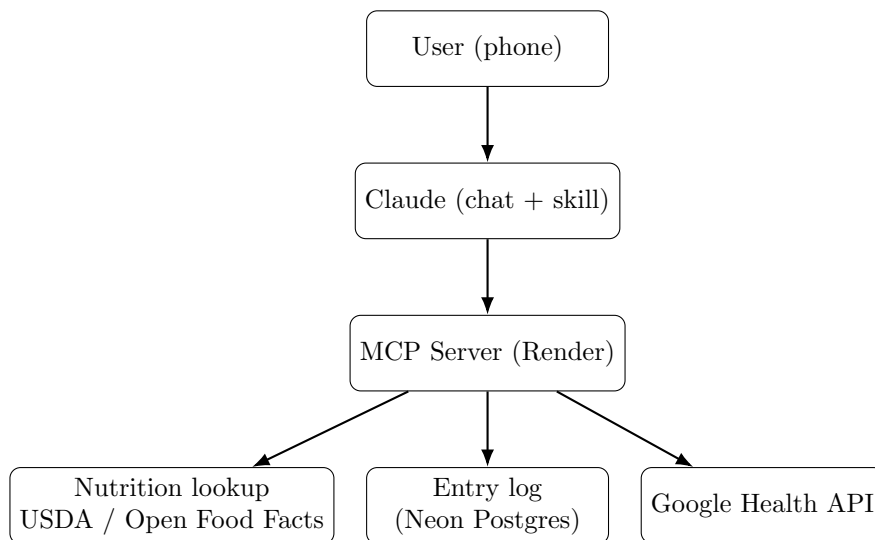
# Claude Fit Logger

## Architecture Document

### 1 Purpose

Claude Fit Logger lets a user log food/drink through natural conversation with Claude, from any device (including mobile), with entries written to Google Health and tracked for undo/removal. It replaces manual food-diary entry with a conversational interface, while keeping nutrition-data lookup automated wherever possible.

### 2 High-Level Flow



### 3 Components

#### 3.1 Claude + Skill Layer

The `calorie-counter` skill governs conversational behavior: filtering zero-calorie drinks, resolving ambiguous portions, maintaining an in-chat running table, and calling the MCP tools. It does not perform its own nutrition lookup except as a fallback when the server reports no match.

#### 3.2 MCP Server (`main.py`)

A Python server built on `fastmcp`, exposing four tools over Streamable HTTP:

- `log_food` — resolves nutrition (automatically or via supplied values) and writes to Google Health + the entry log.

- `undo_entry` — removes the most recent entry from both stores.
- `remove_entry` — removes a named entry from both stores.
- `get_totals` — returns today's running totals.

Authentication is via a Bearer token, satisfied through a minimal self-hosted OAuth 2.1 shim (`oauth.py`) — see §3.6.

### 3.3 Nutrition Lookup (`nutrition_lookup.py`)

Queries USDA FoodData Central first (Foundation/SR Legacy data types preferred, to avoid matching random branded products for generic queries), falling back to Open Food Facts. Returns `None` if neither matches, signalling the caller to estimate manually.

### 3.4 Google Health Client (`health_client.py`)

Wraps the Google Health API (`health.googleapis.com/v4`): token refresh, writing `nutrition-log` data points (anonymous-food entries), and deleting by resource name. This replaced an earlier implementation built against the legacy Google Fit REST API, which was found to write to a different, disconnected data store from the one surfaced in the current Google Health app (see the Troubleshooting document for details of that migration).

### 3.5 Entry Log (`db.py` / Neon Postgres)

A single `entries` table recording every logged item, its nutrient values, and the Google Health `datapoint_name` needed to delete it later. This is what makes `undo/remove` possible, since Google Health does not expose entries by any user-friendly identifier beyond the resource name it returns at creation time.

### 3.6 OAuth Shim (`oauth.py`)

Claude's custom connector implementation requires a remote MCP server to support OAuth 2.1 with Dynamic Client Registration, even for a single-user server with no real multi-user login. This module implements the minimum required surface (discovery metadata, DCR, auto-approved authorize step, token issuance) so the connector integration succeeds, while the underlying credential is simply the server's static API key.

## 4 Data Flow: Logging an Item

1. User states a food item in chat.
2. Skill resolves ambiguity (portion size, variant) if needed.
3. Skill calls `log_food(item_name, quantity_g, tz)`.
4. Server queries USDA, then Open Food Facts.
5. **If found:** nutrients scaled to quantity, written directly.
6. **If not found:** server returns `not_found`; Claude estimates nutrition and re-calls `log_food` with explicit values.
7. Server writes to Google Health, receives a `datapoint_name`.
8. Server inserts a row into Neon (values + `datapoint_name`).
9. Server returns updated running totals; skill renders the chat table.

## 5 Technology Stack

| Layer                   | Choice                                         |
|-------------------------|------------------------------------------------|
| Server framework        | fastmcp (Python) + Starlette + uvicorn         |
| Hosting                 | Render (free web service)                      |
| Database                | Neon (free Postgres, serverless)               |
| Nutrition data          | USDA FoodData Central, Open Food Facts         |
| Health data             | Google Health API v4                           |
| Auth (Google)           | OAuth 2.1, Production-published consent screen |
| Auth (Claude connector) | Self-hosted OAuth 2.1 shim over static API key |

## 6 Known Constraints

- Render’s free tier has no persistent disk and spins down after inactivity (cold starts on first request).
- OAuth client registrations and pending authorization codes in the shim are stored in memory, not Neon — a Render restart clears them, requiring the Claude connector to be reconnected.
- This system is single-user by design; the OAuth shim auto-approves every authorization request without a real login screen.